



Reporte – RPM Price Injector

Price Injector: Movimiento de 03/04
Oracle Retail – Proyecto Oxxo Nucleo

Abril, 2024 | Cliente: Oxxo Mexico
Copyright © 2024, Oracle and/or its affiliates
Confidential – Oracle Restricted

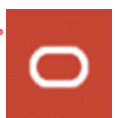
Abril, 2024 | Cliente: Oxxo Mexico
Copyright © 2024, Oracle and/or its affiliates – Confidential: Oracle Restricted

ORACLE
Consulting



TABELA DE CONTENIDO

I.	CONTROL DE VERSIONES.....	3
II.	IDENTIFICACIÓN DEL PRODUCTO.....	3
	Sistemas Involucrados.....	3
III.	ANÁLISIS Y SOLUCIÓN (MOVIMIENTO DE 03/04)	3
	SQL Profiles Creados	3
	Eventos de Espera de Cluster	4
	Alto nivel de Lectura Física sobre tabla de Auditoria	5
	Objetos con Espera en ITL Waits	7
	Script de "Rebuild" y "Shrink Space Cascade"	7
	Incrementar paralelismo de RPM (WEBLOGIC 16 Threads+)	7
	Desactivar "Logger Logs" en la base de datos	8
	Procesos CUSTOM de GAP 36.....	8
	Creación de índices	9
IV.	TIEMPOS DE EJECUCIÓN (MOVIMIENTO DE 03/04).....	10
V.	GLOSARIO	10



I. CONTROL DE VERSIONES

Fecha	Autor	Versión	Revisión
11-Abr-2024	Felipe Donoso / Uilber Lomeu	1.0	Liberación de la versión inicial.

II. IDENTIFICACIÓN DEL PRODUCTO

Producto	Oracle Retail
Dueño de aplicación	
Plataforma	RPM
Lider de Proyecto TI	
Desarrollador	Oracle Consulting Services - Retail

Sistemas Involucrados

RPM – Retail Price Management

III. ANÁLISIS Y SOLUCIÓN (MOVIMIENTO DE 03/04)

Durante la ejecución de las pruebas de inyección de precios resultantes del proceso de movimientos de ubicación, experimentamos algunos contratiempos en las ejecuciones donde se tomaron acciones correctivas conforme se fueron identificando.

A continuación, se presenta un conjunto de puntos identificados que están agregando tiempo innecesario de ejecución al proceso del Inyector, así como sus respectivas acciones tomadas / propuestas.

SQL Profiles Creados

Los siguientes SQL Profiles se han sugerido al equipo de OXXO ser creados para corregir los planes de ejecución que las siguientes consultas han tomado desde el día 03/04/2024. Estos planes de ejecución poseen tiempos más eficientes y menores que los otros planes que se han observado.

Los Profiles nuevo serían los siguientes:

- coe_3fpkgbms4avam_2293484941
- coe_3cpf0mka474db_1891306522
- coe_b04czjcz8zqm9_755290796
- coe_7pcu69c7kysb6_3208337480
- coe_4f8nxxh7s9u6f_1043722706

Con esos planes sugeridos durante las últimas ejecuciones (sábado 06/04) se ve que el proceso de cambio de precio según lo revisado termina en tiempos ya más reducidos. Aquello también fue confirmado en las revisiones realizadas durante la última War Room de dicha día sobre la Tabla de control RMS.RPM_TASK.

NAME	CATEGORY	SIGNATURE	SQL_TEXT	CREATED	LAST_MODIFIED	DESCRIPTION	TYPE	STATUS
coe_3fpkgbms4avam_2293484941	DEFAULT	7.61519138327845E17 (HUGECLOB)	4/4/2024 9:26:29.000000 PM	4/4/2024 9:26:29.000000 PM	coe_3fpkgbms4avam_2293484941_761519138327844751	761519138327844751	MANUAL	ENABLED
coe_3cpf0mka474db_1891306522	DEFAULT	4.51871181976511E17 (HUGECLOB)	4/4/2024 5:59:32.000000 PM	4/4/2024 5:59:32.000000 PM	coe_3cpf0mka474db_1891306522_451871181976510639	451871181976510639	MANUAL	ENABLED
coe_b04czjcz8zqm9_755290796	DEFAULT	1.34537065189826E19 (HUGECLOB)	4/4/2024 3:32:10.000000 PM	4/4/2024 3:32:10.000000 PM	coe_b04czjcz8zqm9_755290796_1345370651898261575	1345370651898261575	MANUAL	ENABLED
coe_7pcu69c7kysb6_3208337480	DEFAULT	6.68274493403787E18 (HUGECLOB)	4/3/2024 2:53:22.000000 PM	4/4/2024 9:53:03.000000 AM	coe_7pcu69c7kysb6_3208337480_6682744934037865346	6682744934037865346	MANUAL	ENABLED
coe_4f8nxxh7s9u6f_1043722706	DEFAULT	3.23084818364478E18 (HUGECLOB)	4/3/2024 8:11:30.000000 PM	4/3/2024 8:11:30.000000 PM	coe_4f8nxxh7s9u6f_1043722706_3230848183644781599	3230848183644781599	MANUAL	ENABLED

Estos SQL Profiles se han sugerido crear luego de ver constantes ejecuciones la semana pasada por parte del Inyector y ver que los Hash Plan Values no eran los mismos que anteriores ejecuciones. Estos SQL Profiles no alteran el comportamiento de otras consultas, sino que sólo mejoran el tiempo de los SQLIDs en específico sobre los cuales se crean.



El comportamiento de los anteriores SQLIDs siempre debe ser revisado y monitoreado en el futuro (al igual que cualquier otra consulta) con el fin de ver si es necesaria una nueva ejecución de Advisor o análisis de Plan de ejecución sobre estas Queries.

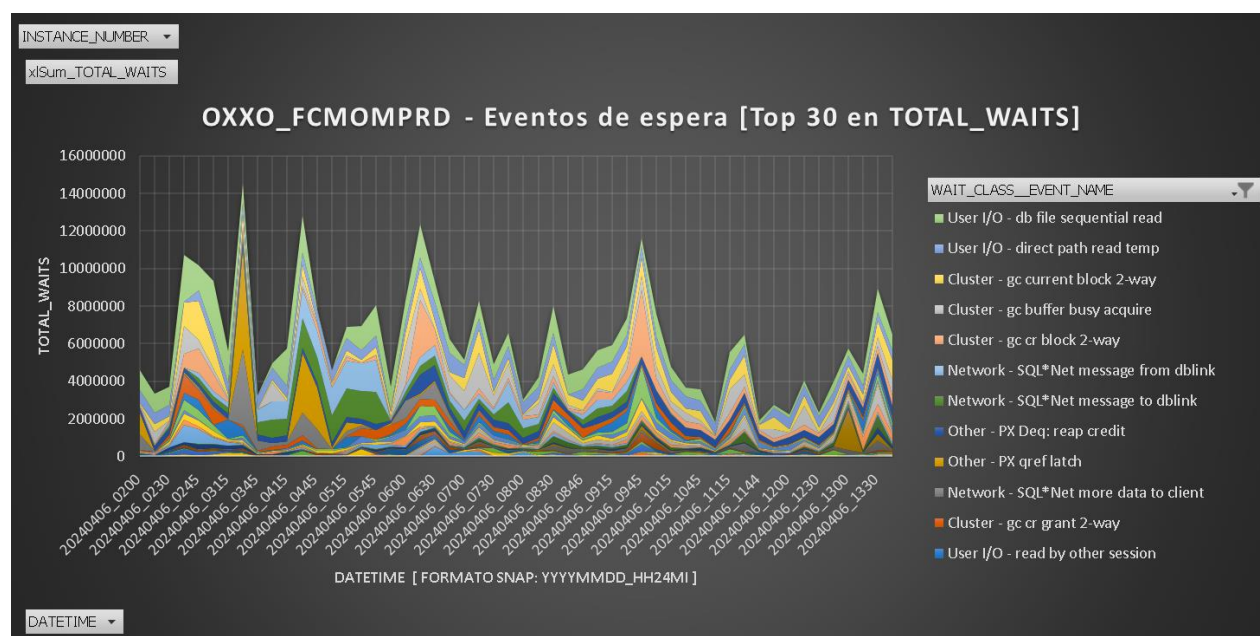
Eventos de Espera de Cluster

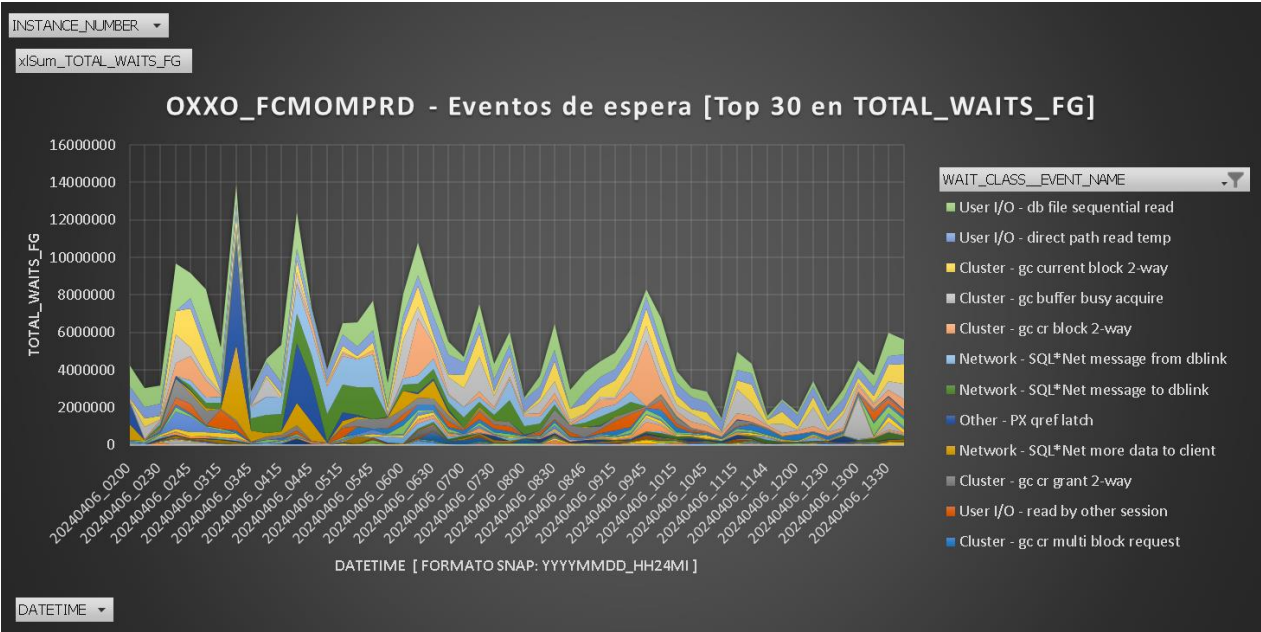
Uno de las Wait Class que más se observa a nivel de base de datos en general (y no solamente durante la ejecución del proceso del Inyector, sino que en general) es la presencia de los eventos "Cluster". En alguna de las War Room de la semana pasada, técnicos especialistas del equipo de Oxxo comentaron la sugerencia de probar la ejecución del Inyector sólo contra una de las instancias de base de datos.

Para probar esto se pueden hacer una de las siguientes alternativas:

- A. Modificar el Data source para apuntar sólo a una de las instancias ejemplos:
VIP_nodo_01:1521/FCMOMPRD1
VIP_nodo_02:1521/FCMOMPRD2
- B. Bien generando un nuevo servicio que sólo se exista en uno de los nodos de Base de Datos y luego haciendo la conexión a dicho servicio a nivel de Data Source, ejemplo:
 1. `srvctl add service -d FCMOMPRD -s FCMOMPRD_INYECTOR_SRV -r instancia_de_BD_FCMOMPRD1_o_FCMOMPRD2 -q TRUE -e SESSION -j LONG -B SERVICE_TIME -m BASIC -z 300 -w 1 -l PRIMARY`
 2. `srvctl start service -d FCMOMPRD -s FCMOMPRD_INYECTOR_SRV`
 3. Luego posteriormente modificar el Datasource a lo siguiente:
oxmomprd-clstr-scan:1521/FCMOMPRD_INYECTOR_SRV

Los siguientes gráficos corresponden a la presencia de los Eventos de espera dónde se aprecian los eventos de Cluster antes mencionados. Está gráfica corresponde al sábado 06/04 dónde el Inyector estuvo corriendo.

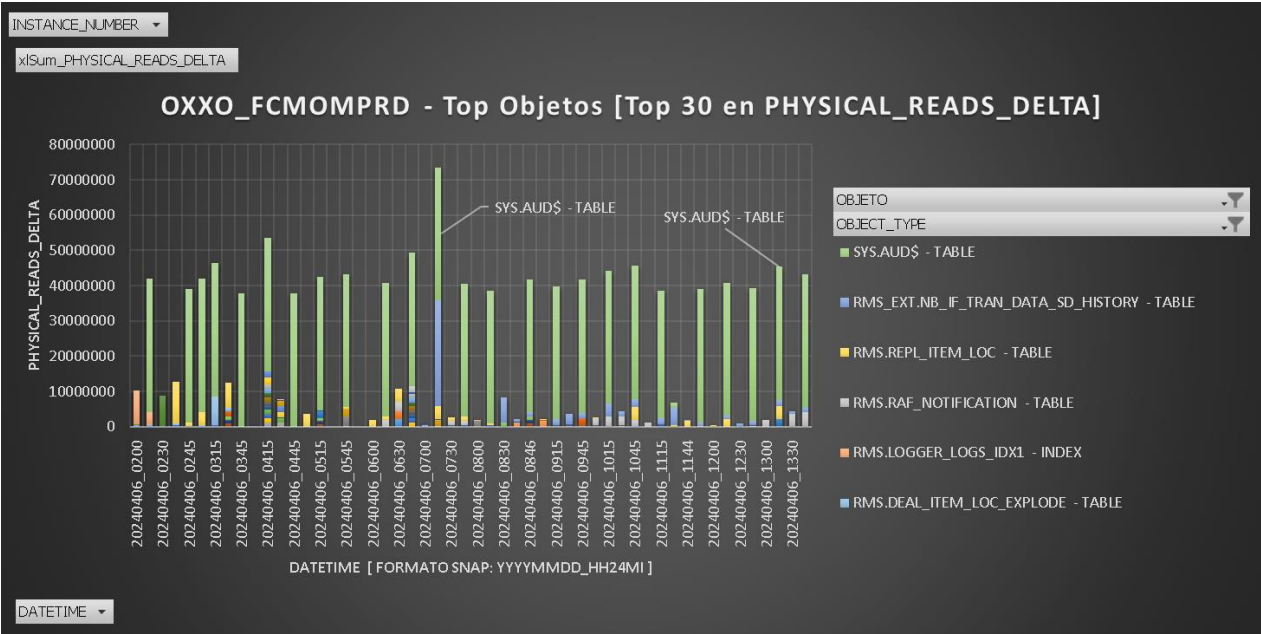




Sugerimos que en alguna futura ejecución esto se pruebe ya que hasta el minuto no se ha realizado acción al respecto.

Alto nivel de Lectura Física sobre tabla de Auditoria

Algo que se aprecia en las gráficas de lecturas física directa sobre disco es un alto nivel de este tipo de lecturas sobre la tabla de auditoria SYS.AUD\$. La gran mayoría de las lecturas **físicas** no son sobre los objetos de RMS si no que más bien sobre la tabla de auditoría. En general esta tabla de auditoria para evitar su crecimiento siempre se recomienda ir haciendo depuración sobre ella, ya sea bien con Deletes de los registros más antiguos o bien truncándola (previo respaldo).



Segment Statistics (Global)

Statistic	Owner	Tablespace Name	Object Name	Subobject Name	Obj. Type	Obj#	Dataobj#	Value	%Total	%Capture
buffer busy waits	SYS	SYSTEM	AUD\$		TABLE	501	1444733	473,051		17.16
logical reads	SYS	SYSTEM	AUD\$		TABLE	501	1444733	2,346,662,400	1.62	6.83
physical reads	SYS	SYSTEM	AUD\$		TABLE	501	1444733	2,344,470,825	32.69	45.65
physical reads direct	SYS	SYSTEM	AUD\$		TABLE	501	1444733	2,344,469,458		46.25
physical read requests	SYS	SYSTEM	AUD\$		TABLE	501	1444733	18,316,334	3.96	19.24

Tamaño actual de la tabla AUD\$ no es menor, es de 288 GB.

```
select owner, segment_name, round(sum(bytes)/1024/1024/1024,2) GB from
dba_segments where segment_name = 'AUD$' and owner = 'SYS'
group by owner, segment_name
```

ata Grid

Messages | Data Grid | Trace | DBMS Output (disabled) | Query Viewer | Explain Plan | Script Output

Drag a column header here to group by that column

OWNER	SEGMENT_NAME	GB
SYS	AUD\$	288.56

Algunas de las consultas que acceden a la AUD\$ por lo que se aprecia son gatilladas por el Enterprise Manager (módulo **Oracle Enterprise Manager.Metric Engine**, usuario DBSNMP)

Drag a column header here to group by that column											
SQL_ID	PARSING_SCHEMA_NAME	MODULE	SQL_PROFILE	OPTIMIZER_COST	EXECUTIONS	ROWS_PROCESSED_XE	BUFFER_GETS_XE	ELAPSED_TIME_XE	CPU_TIME_XE	USER_IO_WAIT_TIME_XE	APPLICATION_WAIT_TIME_XE
44f8fcd8ich	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866677	41.38	5.35		25.22
7k2cv95qaz2m	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866737	40.46	5.34		23.69
2vdv1vka7aa	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866710	41.38	5.54		23.59
0cove8lgr27n7	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866710	41.93	5.58		24.26
0weh116p2hg	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864411	40.22	5.37		23.5
24qek52up4ka	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864411	38.44	5.51		21.22
a0kmc1k7vnt7	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866727	42.34	5.33		25.71
3q3kvq42p47	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866704	43.45	5.55		25.63
9219595dnhh	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866714	41.3	5.37		24.26
4vz49b13246w4	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864412	37.99	5.44		20.97
7m444vqp74.1	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864411	38.76	5.37		22.83
0umrebp4327u	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866673	39.13	5.35		22.36
gm05d62ma3c5	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864411	41.83	5.41		24.92
3pbhqpqazw1	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864493	41.83	5.49		24.55
6d7cpqmdpqn	DBSNMP	Oracle Enterprise Manager Metric Engine		1605054	1439	1	5861316.98	41.12	5.42		24.06
1w1p81uuv05c	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864406	38.84	5.41		21.98
63v45vna92zv	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864411	37.53	5.5		20.42
q12v9c24v84	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864492	42.28	5.46		25.14
1t1g8bdqdlc	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866720	41.3	5.49		23.84
gm74vu43727y	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864486	38.8	5.39		21.99
bg07m5m7u3y	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864525	41.04	5.38		24.14
dh4kmc330e	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5864495	40.29	5.35		23.52
35ad2bdh1pm	DBSNMP	Oracle Enterprise Manager Metric Engine		1604066	1	1	5866710	43.04	5.57		25.18

Por lo que se puede ver esta tabla AUD\$ es accedida por Full table scan en varios de los planes de ejecución de esos SQLIDs, lo que implica un alto costo en el plan de ejecución por el gran número de filas (Recordar que un Full Table Scan no siempre es perjudicial en Tablas con poco volumen, pero si aplica en este caso por el tamaño de 288GB que posee junto con sus 1.732.673.999 de filas):

Plan hash value: 4205270537

Id	Operation	Name	E-Rows	E-Bytes	Cost (%CPU)	E-Time
0	SELECT STATEMENT				1605K(100)	
1	SORT AGGREGATE		1	61		
2	NESTED LOOPS OUTER		523	31903	1605K (2)	00:01:03
3	HASH JOIN RIGHT OUTER		523	29811	1605K (2)	00:01:03
4	INDEX FAST FULL SCAN	I_SYSTEM_PRIVILEGE_MAP	237	1185	2 (0)	00:00:01
5	HASH JOIN RIGHT OUTER		523	27196	1605K (2)	00:01:03
6	INDEX FAST FULL SCAN	I_SYSTEM_PRIVILEGE_MAP	237	1185	2 (0)	00:00:01
7	HASH JOIN		523	24581	1605K (2)	00:01:03
8	INDEX RANGE SCAN	I_AUDIT_ACTIONS	1	20	1 (0)	00:00:01
9	TABLE ACCESS FULL	AUD\$	785K	20M	1605K (2)	00:01:03
10	INDEX RANGE SCAN	I_STMT_AUDIT_OPTION_MAP	1	4	1 (0)	00:00:01

Lo recomendado seria lo siguiente:

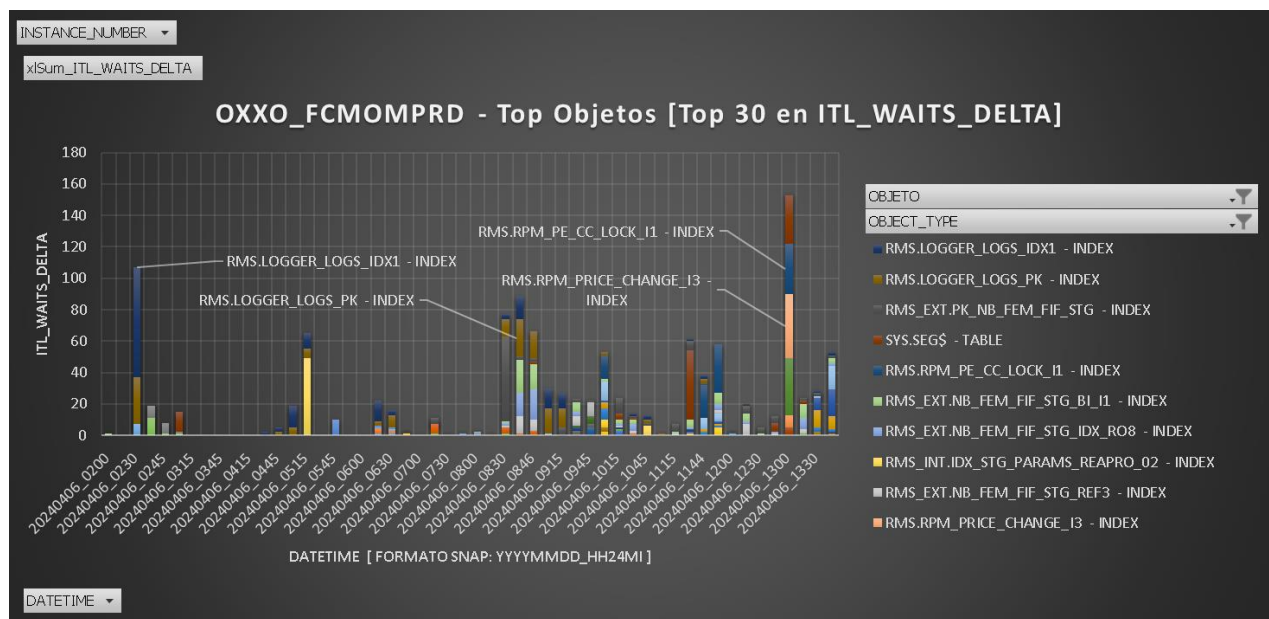
- Si las consultas no son realmente necesarias o de valor seria recomendable desactivarlas por parte del EM.
- Indexar la tabla AUD\$ por el campo de fecha tal como es indicado en la nota *"Performance Issues Caused by DB Collector When AUD\$ Is Very Large (Full Table Scan on AUD\$) (Doc ID 1356088.1)"*
- Otra recomendación seria depurar la tabla eliminando los registros antiguos, para así ir dejando los más recientes o bien haciendo un Truncate a este previo respaldo para que comience nuevamente a ser llenada por las opciones que tienen activadas a nivel de auditoría.



Objetos con Espera en ITL Waits

Durante alguno de los horarios de ejecución de Inyector se visualiza algunos objetos con un nivel que no es muy alto en esperas ITL Waits, pero que por ahora se sugiere que en procesos futuros del Inyector mantengamos un monitoreo sobre esta gráfica para revisar si es necesario realmente una acción sobre estos índices y tablas.

El procedimiento para mitigar esperas de este tipo es como se describe en la nota de soporte *"How To Solve High ITL Waits For Given Segments. (Doc ID 464041.1)"*, dónde se modifican los parámetros de PCTFREE e INITRANS de los objetos junto con su respectivo Rebuild para reducir estas esperas. Como mencionamos este punto se debe seguir monitoreando en el futuro para revisar si es necesario que hagamos alguna sugerencia al respecto.



Script de "Rebuild" y "Shrink Space Cascade"

Se sugiere que antes que comience del proceso de Inyector se ejecute el script de Rebuild index y Shrink space cascade en las tablas de **"Bulk Conflict Check"**. Este script fue enviado por el equipo de Oracle a OXXO.

La recomendación es agregar esta ejecución de script a Control-M antes del proceso de Inyector para quede automático dicho mantenimiento. Esta tarea de reorganización de segmentos es muy recomendable sobre las tablas e índices que sufren constantes y masivos por parte del proceso. Este proceso puede ser ajustado incluyendo más tablas si necesario.

Incrementar paralelismo de RPM (WEBLOGIC 16 Threads+)

Por defecto la instalación de RPM no incluye la implementación de un workmanager en weblogic para manejar el numero de threads que la aplicación de RPM puede utilizar. Es decir que la manera que está, la aplicación de RPM está limitada a utilizar solo 16 threads en sus procesos. Una vez que la máquina trae más recursos disponibles, la alternativa es agregar mas paralelismo a la aplicación para que tenga más poder de ejecución disminuyendo el tiempo total de ejecución. Pues si necesitan mantener activo que sea al nivel "ERROR" y bajo demanda al nivel "DEBUG".



Desactivar “Logger Logs” en la base de datos

El proceso de LOGGER LOGS está permanentemente activa en los ambientes, incluyendo productivo y la recomendación es que sea utilizada puntualmente pues cuando está activa degrada performance de la base de datos una vez que se van generando logs de casi todas las instrucciones de código PLSQL que tiene por objetivo guardar informaciones para depuración de procesos.

Procesos CUSTOM de GAP 36

Fue identificado que una funcionalidad del GAP 36 de cambios de ubicación también afectan el proceso de “Price Injector”, pues tienen tablas NB (custom) que son consultadas y actualizadas en el proceso de “Price Injector” de RPM. Estas partes customizadas se ejecutan en el inicio y al final del job. La función “RPM_PRICE_EVENT_INJECTORSQL.GET_CONFLICT_COUNT” hace una llamada a un código CUSTOM al final del término del proceso core de Injector.

La ejecución del día 06/04 empezó a las 12:52 PM y el proceso core de Injector terminó a las 14:19 PM. La parte CUSTOM de GAP 36 quedó ejecutando hasta las 16:27 PM y se tronó en el ControlM.

Abajo está la clase JAVA referenciada:

```
23 public abstract class InjectorPlsqlOperationCountConflicts extends RpmPlsqlDbOperation {
    static String SQL = "begin ? := RPM_PRICE_EVENT_INJECTOR_SQL.GET_CONFLICT_COUNT(?, ?, ?, ?, ?); end;";

    private int result;

    private Long processId;

    private InjectorPriceEventStatusCode statusToProcess;

    public abstract InjectorPriceEventTypeCode getPriceEventType();
}
```

CUSTOM de GAP36: RMS.RPM_PRICE_EVENT_INJECTOR_SQL

```

/*****
 * MODIFY DATE - 17-JAN-2023
 * MODIFY USER - Ruslan Zakusilov RC
 * TAG - 007
 * DESCRIPTION - Moving all logic from triggers under RPM_STG tables into the packages/
 * and calling them from Injector batch. Reason: performance improvement*/
*****/

FUNCTION GET_CONFLICT_COUNT(o_error_message OUT VARCHAR2,
                           o_conflict_count OUT NUMBER,
                           i_process_id IN NUMBER,
                           i_event_type IN VARCHAR2,
                           i_status_to_process IN VARCHAR2)
RETURN NUMBER IS
    L_program VARCHAR2(50) := 'RPM_PRICE_EVENT_INJECTOR_SQL.GET_CONFLICT_COUNT';

    cursor C_GET_COUNT_PC is...;
    cursor C_GET_COUNT_CL is...;
    cursor C_GET_COUNT CLR is...;
    cursor C_GET_COUNT_SP is...;
    --End 004

    cursor C_GET_COUNT_TP is...;
    --End 004

    cursor C_GET_COUNT FP is...;
    cursor C_GET_COUNT MB is...;
    --End 004

    cursor C_GET_COUNT TRAN is...;

BEGIN
    IF i_event_type = RPM_CONSTANTS.PE_TYPE_PRICE_CHANGE then...;

    --Begin 007 Call function for updating statuses into NB_SVC tables
    IF NB_RPM_INJECTOR_CUSTOM_LOGIC.UPDATE_NB_RPM_SVC_TABLES(o_error_message,
                                                            NULL,
                                                            i_process_id,
                                                            NULL, --I_bulk_cc_pe_id
                                                            i_event_type,
                                                            i_status_to_process,
                                                            1) != 1 THEN
        -- I-Get records are sent to Bulk tables
        logger.log('NB_RPM_INJECTOR_CUSTOM_LOGIC.UPDATE_NB_RPM_SVC_TABLES ERROR: ' || o_error_message || ' I_process_id:' || i_process_id);
        return 0;
    END IF;
    --End 007

    return 1;

EXCEPTION
    when OTHERS then
        o_error_message := SQL_LIB.CREATE_MSG('PACKAGE_ERROR',
                                              SQLERRM,
                                              L_program,
                                              TO_CHAR(SQLCODE));

        return 0;

END GET_CONFLICT_COUNT;
```



Creación de índices

Fue sugerido por DBA de OXXO durante una ejecución la creación de un índice en la tabla RPM_FUTURE_RETAIL_GTT, pues se estaba generando un FULL TABLE ACCESS en una operación que se ejecutaba muchas veces y que mejorando este punto la operación final disminuiría un poco de tiempo en el total de la operación.

Sin el índice:

Índice creado:

`create index rms.RPM_FUTURE_RETAIL_GTT_I6 on rms.RPM_FUTURE_RETAIL_GTT(FUTURE_RETAIL_ID);`

Con el índice:



IV. TIEMPOS DE EJECUCIÓN (MOVIMIENTO DE 03/04)

Con las acciones aplicadas en el día 06/04 tuvimos un promedio de ejecución de:

100K → 09 minutos total (promedio de 3 minutos y 40 segundos por thread)

670K → 01 hora y 27 minutos total (promedio de 16 minutos y 20 segundos por thread)

Fue posible notar que el volumen total de cambios de precios en la tabla stage impacta directamente el tiempo de ejecución de cada thread. Cuanto mayor el volumen total en la tabla stage, más se tarda la ejecución de cada thread, mismo que cada thread tenga siempre la misma carga de ítem/loc.

Nuevas pruebas en el ambiente de PRF serán hechas bajando el valor de LUW_COUNT de 10000 para 1000, una vez que esta operación se hace con cambios de precios que tienen solo 1 artículo y 1 tienda.

V. GLOSARIO

Acrónimo	Descripción
RMS	Oracle Retail Merchandising System
RIB	Oracle Retail Integration Bus
MOM	Oracle Retail Merchandising Operations Management
STG	Stage, tablas intermedias
DDL	Data Definition Language

